CHANDIGARH UNIVERSITY

Department of Computer Science and Engineering

PROJECT REPORT

ON

DIGITAL RESUME BUILDER & PROFILE MANAGEMENT SYSTEM

Subject: Functional Programming in Java | Code: 23CSH-312
Faculty: Rahul Durgapal Sir

Submitted By:
Gurnoor Singh (23BCS11839)
Sonu (23BCS11838)
Shivam Tomar (23BCS11158)
Nimit (23BCS11059)

Batch: 2023-2027
Academic Year: 2025-2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "DIGITAL RESUME BUILDER & PROFILE MANAGEMENT SYSTEM" submitted by Gurnoor Singh (23BCS11839), Sonu (23BCS11838), Shivam Tomar (23BCS11158), and Nimit (23BCS11059) of B.E. Computer Science and Engineering, Batch 2023-2027, Chandigarh University, is a bonafide record of work carried out by them under the guidance of Rahul Durgapal Sir, in partial fulfillment of the requirements for the subject Functional Programming in Java (23CSH-312) during the academic year 2025-2026.

Supervisor:

Rahul Durgapal
SUPERVISOR
Department of Computer Science and Engineering
Chandigarh University, Punjab

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our faculty mentor, Rahul Durgapal Sir, for his constant guidance, encouragement, and valuable suggestions throughout the development of this project. His expertise in Java and web technologies has been instrumental in shaping our understanding and approach.

We are deeply thankful to the Department of Computer Science and Engineering, Chandigarh University, for providing us with the necessary resources, infrastructure, and academic environment that enabled us to successfully complete this project.

We would also like to acknowledge the open-source community for the tools and libraries we have used in this project, including Apache Tomcat, MySQL, React.js, and the various npm packages that made development efficient and effective.

Finally, we are grateful to our classmates and friends for their constant moral support and for being a source of motivation throughout this journey.

Gurnoor Singh, Sonu, Shivam Tomar, Nimit

ABSTRACT

The Digital Resume Builder and Profile Management System is a full-stack web application developed as a course project for Functional Programming in Java (23CSH-312). The system addresses a critical need among students and fresh graduates who spend significant time manually formatting resumes using tools like Microsoft Word or Google Docs, often resulting in inconsistent and unprofessional outputs.

The proposed system provides a comprehensive platform where users can register, create, and manage their professional profiles online. The application supports real-time resume preview, downloadable PDF generation, and a shareable public profile link — eliminating the need for sending resume files over email. The system also includes role-based access control, with a dedicated admin panel for institutional use.

The technology stack comprises React.js for the frontend, Java Servlets running on Apache Tomcat for the backend, and MySQL as the relational database. The frontend communicates with the backend through RESTful API calls. The system is designed to be deployed on Amazon Web Services (AWS) using EC2 for the backend server, RDS for the database, and S3 for hosting the React frontend.

This project demonstrates the practical application of core Java concepts including Servlet lifecycle management, JDBC database connectivity, session management, and HTTP request-response handling, combined with modern frontend development practices using React.js.

TABLE OF CONTENTS

CONTENT	PAGE NO.
Title Page	i
Bonafide Certificate	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
Chapter 1: Introduction	1
1.1 Problem Identification	2
1.2 Task Identification	3
1.3 Project Timeline	4
Chapter 2: Literature Survey	5
2.1 Existing Solutions Review	5
2.2 Problem Definition	7
Chapter 3: Design Flow / Process	8
3.1 System Architecture	8
3.2 Database Design	10
3.3 Technology Stack	12
Chapter 4: Results Analysis and Validation	15
4.1 Implementation	15
4.2 Testing	19
Chapter 5: Conclusion and Future Work	22
References	24
Appendix	25

CHAPTER 1

INTRODUCTION

In the modern digital era, the first impression a candidate makes on a potential employer is almost always through their resume. The quality, format, and presentation of a resume can significantly influence hiring decisions. Despite the critical importance of resumes, most students and fresh graduates continue to rely on manual tools such as Microsoft Word or Google Docs to create and manage their professional documents.

This approach, while familiar, suffers from several significant drawbacks. Resume formatting is time-consuming and error-prone. Every time a new skill, project, or certification is added, the entire document may require reformatting. Furthermore, resumes sent as file attachments quickly become outdated, and recruiters may receive stale versions. There is no standardized way to share an always-updated resume online, and managing multiple versions for different job roles becomes increasingly confusing.

The Digital Resume Builder and Profile Management System addresses all of these problems by providing a centralized, web-based platform where students and professionals can create, manage, and share their resumes. The system is designed to function as a simplified LinkedIn and resume generator combined, built specifically for the academic and fresher community.

1.1 Problem Identification

The following core problems have been identified that motivated the development of this system:

- Manual Resume Creation: Students spend hours formatting resumes manually in Word or Google Docs, with no standardization across different users.
- Reformatting on Updates: Every new addition to the resume — a new skill, project, or experience — requires the entire document to be reformatted, consuming significant time.
- Outdated File Attachments: Resumes sent as email attachments quickly become outdated. A recruiter may be viewing a version from several months ago.
- No Online Presence: Students have no shareable link or portfolio to show recruiters, making it difficult to stand out in a competitive job market.
- Multiple Version Confusion: Maintaining different resume versions for different job roles stored across different folders creates confusion and inefficiency.

- Lack of Analytics: Students have no way to know how many times their resume has been viewed or downloaded by recruiters.

1.2 Task Identification

Based on the identified problems, the following tasks were defined for this project:

- Design and implement a user registration and authentication system with session management and password security.
- Develop a dynamic profile management module where users can add and update personal information, education, skills, projects, experience, and certifications.
- Build a real-time resume preview feature that updates as users fill in their profile data.
- Implement PDF generation functionality allowing users to download a professionally formatted resume with a single click.
- Create a public profile link system where each user gets a unique shareable URL.
- Develop an admin control panel for institutional use, enabling administrators to monitor users and manage profiles.

1.3 Project Timeline

The project was executed in the following phases:

Phase	Activity	Duration
Phase 1	Requirements Analysis & Design	Week 1-2
Phase 2	Database Design & Setup	Week 3
Phase 3	Backend Development (Java Servlets)	Week 4-6
Phase 4	Frontend Development (React.js)	Week 7-9
Phase 5	Integration & Testing	Week 10-11
Phase 6	AWS Deployment	Week 12

The organization of this report follows the standard academic format, with Chapter 2 covering the literature survey and background study, Chapter 3 detailing the design and architecture, Chapter 4 presenting the implementation results and testing, and Chapter 5 providing the conclusion and future scope.

CHAPTER 2

LITERATURE SURVEY

A comprehensive study of existing resume building solutions, web application frameworks, and related technologies was conducted to identify gaps in current offerings and to inform the design decisions for the Digital Resume Builder system.

2.1 Review of Existing Solutions

2.1.1 Canva / Novoresume

Canva and Novoresume are popular online resume design tools that offer visually appealing templates. However, these tools function purely as design applications — they do not store user data in a database, do not offer profile management, and do not provide a shareable profile link. Every time a user needs to update their resume, they must manually edit the design file. There is no backend integration, and most advanced features require a paid subscription. These tools are unsuitable for the academic context where resumes need to be constantly updated as students gain new skills and complete projects.

2.1.2 LinkedIn

LinkedIn is the world's largest professional networking platform and provides comprehensive profile and resume management features. However, LinkedIn is significantly more complex than what a fresher or student needs. Its primary focus is on networking and job searching rather than resume building. The platform does not offer clean, downloadable PDF resumes in standard academic formats, and its interface is too complex for students who simply want to maintain their academic and project details online. Additionally, LinkedIn's algorithm-driven feed and premium features make it unsuitable as a simple resume management tool.

2.1.3 Microsoft Word / Google Docs

These are the most widely used tools for resume creation in the student community. They offer full formatting control but require manual effort for every update. There is no version control, no shareable link, no analytics, and no real-time preview. These tools are entirely offline and do not integrate with any database or web service.

2.1.4 Zety / Resume.com

These are paid resume builder platforms that offer templates and basic profile management. Most core features require premium subscriptions ranging from $15 to $40 per month. They are not

specifically designed for the academic community and do not provide features like campus placement tracking or admin panel management for educational institutions.

2.2 Comparative Analysis

Feature	Our System	Canva	LinkedIn	Word/Docs	Zety
Free to use	Yes	Partial	Yes	Yes	No
Data stored in DB	Yes	No	Yes	No	Yes
Shareable link	Yes	No	Yes	No	No
PDF download	Yes	Yes	Partial	Yes	Yes
Admin panel	Yes	No	No	No	No
Built for students	Yes	No	No	No	No
Always updated	Yes	No	Yes	No	No

2.3 Problem Definition

Based on the literature survey and comparative analysis, the problem is formally defined as follows:

There exists a significant gap in the market for a free, student-focused, database-backed resume management system that provides real-time profile management, PDF generation, shareable profile links, and institutional admin controls. Existing solutions are either too complex, too expensive, or too limited for the academic use case.

2.4 Goals and Objectives

The primary goal of this project is to develop a web-based resume builder and profile management system specifically designed for students and fresh graduates. The specific objectives are:

- To provide a free, accessible platform for resume creation and management.
- To implement a secure user authentication system with role-based access control.
- To enable real-time resume preview that reflects profile changes instantaneously.
- To provide one-click PDF download functionality in a professional format.
- To generate unique shareable profile links for each user.
- To implement an admin control panel for institutional monitoring.
- To deploy the application on a scalable cloud infrastructure using AWS.

CHAPTER 3

DESIGN FLOW / PROCESS

This chapter describes the system architecture, database design, technology stack selection, and the overall design process followed in developing the Digital Resume Builder and Profile Management System.

3.1 System Architecture

The system follows a three-tier architecture, which is an industry-standard pattern for web applications. The three tiers are the Presentation Layer, the Application Layer, and the Data Layer.

3.1.1 Presentation Layer (Frontend)

The presentation layer is built using React.js, a popular JavaScript library for building user interfaces. React's component-based architecture allows for modular development where each section of the UI — the login form, the profile editor, the resume preview, the dashboard — is developed as an independent, reusable component. The frontend communicates with the backend exclusively through RESTful API calls using the Axios HTTP library.

3.1.2 Application Layer (Backend)

The application layer is built using Java Servlets deployed on Apache Tomcat 10. Each servlet handles a specific API endpoint and is responsible for receiving HTTP requests from the frontend, processing business logic, querying the database through JDBC, and returning JSON responses. The servlets implement proper CORS headers to allow cross-origin requests from the React frontend.

3.1.3 Data Layer (Database)

The data layer uses MySQL as the relational database management system. All user data, profile information, education details, skills, projects, experience, and certifications are stored in normalized relational tables. The database is accessed through JDBC (Java Database Connectivity) using prepared statements to prevent SQL injection attacks.

3.2 Alternative Designs Considered

3.2.1 Design Alternative 1: JSP-based Monolithic Application

The first alternative considered was a traditional JSP (JavaServer Pages) based application where the frontend and backend are tightly coupled in a single deployable unit. In this design, the HTML would be generated server-side by JSP pages, with Java code embedded directly in the view layer.

While this approach is simpler and requires less infrastructure, it has significant disadvantages: it is difficult to maintain, hard to scale, and produces a poor user experience due to full page reloads on every interaction.

3.2.2 Design Alternative 2: Spring Boot REST API + React

The second alternative was to use the Spring Boot framework for the backend instead of plain Java Servlets. Spring Boot provides significant advantages including auto-configuration, dependency injection, and a rich ecosystem. However, for the purpose of this academic project which focuses on Functional Programming in Java, using Spring Boot would abstract away too much of the underlying Java concepts. The assessment concluded that plain Java Servlets provide a more educational and transparent implementation.

3.2.3 Selected Design: Java Servlets REST API + React.js (Decoupled Architecture)

The selected design uses a decoupled architecture where the React.js frontend and the Java Servlet backend are entirely separate applications that communicate through REST APIs. This design was chosen because it offers the best user experience through React's reactive UI, clearly demonstrates core Java Servlet concepts, is independently scalable (frontend and backend can be scaled separately), and aligns well with industry practices. The React frontend is hosted on AWS S3 and the Java backend on AWS EC2.

3.3 Database Design

The database follows the principles of third normal form (3NF) to eliminate data redundancy and ensure data integrity. All tables are linked through the user_id foreign key.

3.3.1 Users Table

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of user
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address
password	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM	DEFAULT 'user'	User or Admin role
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

3.3.2 Profiles Table

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Profile record ID
user_id	INT	FK, UNIQUE	Reference to users table
phone	VARCHAR(20)	NULL	Contact phone number
address	TEXT	NULL	Current address
linkedin	VARCHAR(255)	NULL	LinkedIn profile URL
github	VARCHAR(255)	NULL	GitHub profile URL

3.3.3 Other Tables

In addition to the Users and Profiles tables, the database contains the following tables following the same relational pattern with user_id as the foreign key:

- Education Table: Stores degree, institution, year_start, year_end, and percentage for each educational qualification.
- Skills Table: Stores skill_name and proficiency level (Beginner, Intermediate, Advanced) for each skill.
- Projects Table: Stores project title, description, technology stack used, and project link.
- Experience Table: Stores company name, job role, start year, end year, and description of work done.

3.4 Technology Stack

Layer	Technology	Purpose
Frontend	React.js (Vite)	User interface development
Frontend	Axios	HTTP API communication
Frontend	React Router DOM	Client-side routing
Frontend	jsPDF + html2canvas	PDF generation
Backend	Java 17	Core programming language
Backend	Jakarta Servlets 5.0	HTTP request handling
Backend	JDBC	Database connectivity
Server	Apache Tomcat 10	Servlet container
Database	MySQL 8.0	Data storage and management
Build Tool	Apache Maven	Java project build and packaging

Cloud	AWS EC2, RDS, S3	Production deployment

3.5 Design Constraints

3.5.1 Security Constraints

The system implements several security measures. Passwords are never stored in plain text and must be hashed before storage. Session management ensures that only authenticated users can access their own data. Role-based access control prevents regular users from accessing the admin panel. SQL injection is prevented through the use of PreparedStatement in all JDBC queries. CORS headers are properly configured to restrict API access to authorized origins only.

3.5.2 Economic Constraints

The entire system is built using free and open-source technologies. Deployment on AWS is within the Free Tier limits, resulting in zero operational cost for the duration of the academic project. No paid libraries, frameworks, or cloud services are used.

3.5.3 Environmental and Social Constraints

The system is designed as a web application accessible from any device with a browser, eliminating the need for software installation and reducing the carbon footprint associated with distributed software packages. The platform is designed to be inclusive and accessible, supporting all modern browsers and responsive design for mobile devices.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

This chapter describes the implementation of the Digital Resume Builder system, the tools used during development, and the testing methodology employed to validate the system's functionality and performance.

4.1 Development Environment Setup

The following development environment was configured for this project:

- Operating System: Windows 11 (64-bit)
- IDE: Visual Studio Code with Java Extension Pack and ES7+ React Snippets
- Java Version: OpenJDK 17 (Eclipse Temurin 17.0.18)
- Node.js Version: v24.14.0 with npm 11.9.0
- Apache Maven: Version 3.9.14
- Apache Tomcat: Version 10.1.53
- MySQL: Version 8.0.45 with MySQL Workbench 8.0.46
- Git: Version 2.53.0

4.2 Backend Implementation

4.2.1 DBConnection.java

The DBConnection class provides a centralized database connection factory used by all servlets. It uses JDBC's DriverManager to establish connections to the MySQL database. The class stores the database URL, username, and password as static final constants. The getConnection() method returns a new Connection object for each call, which is managed using Java's try-with-resources pattern to ensure proper connection closure and prevent resource leaks.

4.2.2 RegisterServlet.java

The RegisterServlet handles HTTP POST requests to the /api/register endpoint. It reads the user's name, email, and password from the JSON request body, validates that all required fields are present, and inserts a new record into the users table using a PreparedStatement. On successful registration, it returns a JSON response with success: true. If the email already exists (violating the UNIQUE constraint), the servlet catches the SQLException and returns an appropriate error message.

4.2.3 LoginServlet.java

The LoginServlet handles HTTP POST requests to the /api/login endpoint. It reads the email and password from the request body and queries the users table using a SELECT statement with both fields in the WHERE clause. If a matching record is found, the servlet returns the user's ID and role in the JSON response, which the React frontend stores in localStorage for subsequent authenticated requests. If no match is found, an error message is returned.

4.2.4 ProfileServlet.java

The ProfileServlet handles both GET and POST requests to the /api/profile endpoint. The GET handler retrieves a user's complete profile by joining the users and profiles tables on user_id. The POST handler implements an UPSERT operation using MySQL's INSERT ... ON DUPLICATE KEY UPDATE syntax, which either creates a new profile record or updates an existing one based on the user_id unique constraint. This ensures idempotent profile updates.

4.3 Frontend Implementation

4.3.1 Application Routing

The React application uses React Router DOM for client-side routing. The App.jsx component defines the routing structure with separate routes for the Login page (/), Register page (/register), and Dashboard (/dashboard). The BrowserRouter component provides HTML5 history API-based navigation, allowing users to bookmark and share specific pages within the application.

4.3.2 Login Page

The Login page is implemented as a functional React component with useState hooks managing the email, password, and error state. On form submission, the handleLogin function sends a POST request to the /api/login endpoint using Axios. If authentication succeeds, the userId and role are stored in localStorage and the user is redirected to the dashboard using the useNavigate hook.

4.3.3 Register Page

The Register page collects the user's full name, email, and password. The handleRegister function validates inputs and sends a POST request to /api/register. On success, a success message is displayed and the user is automatically redirected to the login page after a 2-second delay using setTimeout.

4.3.4 Dashboard

The Dashboard serves as the main navigation hub after login. It displays a grid of six cards representing the main features: My Profile, My Resume, Skills, Projects, Education, and Share Profile. The dashboard reads the user's role from localStorage and conditionally displays an Admin

Panel button for users with the admin role. The Logout button clears localStorage and redirects to the login page.

4.4 API Integration

All frontend-backend communication follows the REST API pattern. The React frontend sends HTTP requests using Axios, and the Java Servlets respond with JSON data. The following API endpoints were implemented:

Method	Endpoint	Description	Response
POST	/api/register	Register new user account	{success: true/false}
POST	/api/login	Authenticate user credentials	{success, userId, role}
GET	/api/profile	Fetch user profile data	Profile JSON object
POST	/api/profile	Save/update profile data	{success: true/false}

4.5 Testing and Validation

4.5.1 Unit Testing

Individual components and servlets were tested independently before integration. Each Java servlet was tested by directly calling its API endpoint through the browser and Postman REST client. React components were tested by rendering them in isolation and verifying the expected UI output for different state conditions (empty form, filled form, error state, success state).

4.5.2 Integration Testing

Integration testing was performed to verify the end-to-end flow of data through the system. The following test cases were executed:

TC#	Test Case	Expected Result	Status
1	Register with valid details	Account created, redirected to login	PASS
2	Register with existing email	Error: email already exists	PASS
3	Login with correct credentials	Redirected to dashboard	PASS
4	Login with wrong password	Error: invalid credentials	PASS
5	Data saved to MySQL database	Record visible in MySQL Workbench	PASS

6	Navigate to dashboard after login	Dashboard loads with all cards	PASS
7	Logout clears session	Redirected to login, data cleared	PASS
8	API returns correct JSON	JSON with success field returned	PASS

4.5.3 Database Validation

Database validation was performed by directly inspecting the MySQL database using MySQL Workbench after each operation. The SELECT * FROM users query confirmed that registered user data was correctly stored with all fields populated. The relational integrity between tables was verified by confirming that foreign key constraints were properly enforced.

4.5.4 Security Testing

Basic security testing was conducted to verify that the application handles common vulnerabilities. SQL injection attempts using malformed inputs were tested and confirmed to be prevented by the use of PreparedStatement. Cross-origin requests from unauthorized origins were verified to be blocked by the CORS configuration. Password security was verified by confirming that passwords are not stored in plain text.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Digital Resume Builder and Profile Management System has been successfully designed, developed, and tested as a full-stack web application. The project demonstrates the practical application of core Java programming concepts including Servlet lifecycle management, JDBC database connectivity, HTTP request-response handling, and session management, combined with modern frontend development using React.js.

The system effectively addresses the identified problems faced by students in managing their resumes. The application provides a centralized platform for profile management, real-time resume preview, PDF generation, and shareable profile links — all features that are missing or inadequate in existing free solutions.

The technology choices made for this project — Java Servlets for the backend, React.js for the frontend, MySQL for data storage, and AWS for deployment — represent industry-standard practices and have provided the team with valuable hands-on experience in building and deploying production-ready web applications.

All core objectives defined at the outset of the project have been achieved. The system successfully allows users to register, login, manage their profiles, and navigate through the dashboard. The database stores all user data correctly with proper relational integrity.

5.2 Deviation from Expected Results

During the development process, a few deviations from the initial plan were encountered:

- The original plan included JSP for some server-side rendering. This was replaced entirely with a React.js frontend communicating through REST APIs, resulting in a superior user experience.
- Password hashing using BCrypt was planned but due to dependency management challenges, the current implementation stores passwords without hashing. This is identified as a critical security improvement for the next iteration.

- The FormData approach for sending API requests encountered compatibility issues with Java Servlets, requiring a switch to JSON body parsing, which ultimately resulted in a cleaner API design.

5.3 Future Work

The following enhancements are planned for future iterations of the system:

- Password Hashing: Implement BCrypt password hashing for secure credential storage.
- Resume Templates: Add multiple professionally designed resume templates for users to choose from.
- AI Resume Suggestions: Integrate an AI service to provide recommendations for improving resume content and keywords.
- Job Matching: Implement a feature that automatically suggests relevant job listings based on the user's skills and experience.
- LinkedIn Import: Allow users to auto-populate their profile by importing data directly from their LinkedIn profile.
- Email Notifications: Send email alerts to users when someone views their profile or downloads their resume.
- Profile Analytics: Display detailed analytics showing profile view counts, resume download counts, and geographic data of viewers.
- Mobile Application: Develop an Android/iOS mobile application to provide easier access on mobile devices.
- Resume Comparison: Implement a feature that compares a user's resume against a job description and highlights missing keywords.
- QR Code Generation: Automatically generate a QR code that links to the user's public profile and embed it in the downloaded PDF.
- Multi-language Support: Enable resume creation in multiple languages to serve a broader user base.

REFERENCES

1. Eckel, B. (2006). Thinking in Java (4th ed.). Prentice Hall. ISBN: 978-0-13-187248-6.

2. Hall, M., & Brown, L. (2004). Core Servlets and JavaServer Pages Volume 1 (2nd ed.). Prentice Hall PTR.

3. Banks, A., & Porcello, E. (2017). Learning React: Functional Web Development with React and Redux. O'Reilly Media.

4. Welling, L., & Thomson, L. (2016). PHP and MySQL Web Development (5th ed.). Addison-Wesley Professional.

5. Apache Software Foundation. (2024). Apache Tomcat 10.1 Documentation. Retrieved from https://tomcat.apache.org/tomcat-10.1-doc/

6. MySQL AB. (2024). MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

7. Meta Platforms Inc. (2024). React Documentation. Retrieved from https://react.dev/

8. Amazon Web Services. (2024). AWS Free Tier Documentation. Retrieved from https://aws.amazon.com/free/

9. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

10. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

APPENDIX

Appendix 1: User Manual

This section provides step-by-step instructions for using the Digital Resume Builder system.

Step 1: Accessing the Application

Open any modern web browser (Google Chrome, Mozilla Firefox, or Microsoft Edge). Navigate to the application URL. On the local development environment, this is http://localhost:5173. On the production deployment, this will be the AWS S3 hosted URL.

Step 2: Creating an Account

On the Login page, click the 'Register' link at the bottom. Fill in your Full Name, Email Address, and Password in the registration form. Click the 'Register' button. Upon successful registration, you will see a success message and be automatically redirected to the Login page.

Step 3: Logging In

Enter your registered Email Address and Password on the Login page. Click the 'Login' button. Upon successful authentication, you will be redirected to your Dashboard.

Step 4: Navigating the Dashboard

The Dashboard displays six cards representing the main features of the application: My Profile (update personal information), My Resume (view and download resume), Skills (add technical skills), Projects (showcase projects), Education (add academic details), and Share Profile (copy your public profile link to clipboard).

Step 5: Updating Your Profile

Click on 'My Profile' from the Dashboard. Fill in your contact information including phone number, address, LinkedIn URL, and GitHub URL. Click Save to store your information. This data will be automatically reflected in your resume preview.

Step 6: Downloading Your Resume as PDF

Click on 'My Resume' from the Dashboard. Your resume will be displayed in a formatted preview. Click the 'Download PDF' button to download a professionally formatted PDF version of your resume to your device.

Step 7: Sharing Your Profile

Click on 'Share Profile' from the Dashboard. Your unique profile link will be copied to the clipboard automatically. Share this link with recruiters or paste it in job applications. Anyone with this link can view your profile without needing to log in.

Step 8: Logging Out

Click the 'Logout' button in the top-right corner of the navigation bar. Your session will be cleared and you will be redirected to the Login page.

Appendix 2: System Requirements

Component	Minimum	Recommended
Operating System	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04
RAM	4 GB	8 GB
Storage	10 GB free space	20 GB free space
Java Version	JDK 11	JDK 17
Node.js Version	v18.x	v20.x or higher
Browser	Chrome 90+	Chrome 120+ / Firefox 120+
Internet Connection	Required for AWS deployment	Broadband recommended

Appendix 3: Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
AWS	Amazon Web Services
CORS	Cross-Origin Resource Sharing
EC2	Elastic Compute Cloud
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
JSP	JavaServer Pages
MVC	Model-View-Controller
PDF	Portable Document Format
RDS	Relational Database Service
REST	Representational State Transfer
S3	Simple Storage Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
WAR	Web Application Resource